

Olist Reconciliation Project: Study Guide

Personal study guide: how the project works, why it was built this way, and how to explain it.

START HERE: the whole project in 5 sentences

Learn these 5 sentences first. Say them out loud until you do not need to look. Everything else in this guide is detail on top of these.

1. A shop has two lists about the same orders. **List A (order items)** says what customers bought. **List B (payments)** says what customers paid. They should match, but they do not always, and nobody trusts the numbers.
2. For each of the 99,441 orders I added up A and B and compared them. If they differ by more than 1 cent (R\$ 0.01), it is a **mismatch**.
3. 98.91% of orders matched. 1,079 did not.
4. The big problem: **R\$ 162,591.95** was paid on 775 orders that have no items recorded. 98.97% of those orders were canceled or unavailable, so the question is whether those customers got their money back.
5. I never deleted bad data. Every bad row goes into an **exceptions table** with a reason and a severity, so nothing disappears and everything can be audited.

All money is in Brazilian real (R\$, BRL), not dollars.

Why this project is worth something

Most analyst portfolios are a notebook on a clean dataset. This one does what real analysts get paid for: two systems disagree, you find out why, you put an amount in reais on it, and you tell a manager what to do. The pipeline is also automatic: one command rebuilds everything, and tests prove the checks work.

Words you need (plain English)

Word	What it means
Reconciliation	Comparing two records of the same thing and explaining every difference.
System A / System B	A = order items (what was sold). B = payments (what was paid).
Gap	Paid minus sold, per order. Positive = customer paid more than the items cost.
Tolerance	How big a gap we ignore. Here: 1 cent.
Match	Gap is within tolerance.
Bucket / category	The reason we assign to a mismatch (rounding, interest, and so on). Each order gets exactly one.
Exception	A row that broke a rule. Logged in the exceptions table with a reason and severity.
Severity	high = money or counts are wrong if you trust the row. medium = suspicious. low = just worth knowing.
Quarantine	Keep the row, but leave it out of the clean tables. Never delete.
Schema contract	A written list of what each table must look like (columns, types, required fields, allowed values). Ours is <code>config/contracts.yaml</code> .
Orphan	A child row with no parent, for example a payment for an order that does not exist.
Foreign key	A column that points to a row in another table (payment.order_id points to orders.order_id).
CTE	A named step inside a SQL query (<code>WITH step AS (...)</code>). Makes long queries readable.
Window function	A SQL calculation across related rows without collapsing them, like ranking the biggest gap per month.
DECIMAL	A number type that keeps cents exact. Normal floating point numbers drift.
DuckDB	A fast SQL database that runs inside Python, no server needed.
Star schema	Tables arranged for dashboards: a few long "fact" tables (events) linked to short "dimension" tables (dates, states, categories).
Parquet	A compact file format for tables. Opens fast in Power BI.
MAPE	Mean absolute percentage error. How wrong a forecast is, as a percent. Lower is better.
Baseline (naive forecast)	The dumb forecast, like "next month equals last month". A real model must beat it.
Holdout	Months you hide from the model, then use to grade it.
pytest	The Python tool that runs the tests.
Fixture	A small sample of real data used by tests, so they run in seconds.
Failure injection	Planting a known bad record in a test copy of the data to prove the check catches it.
CI	Continuous integration. GitHub runs the tests automatically on every push.

The flow: what happens when you run one command

`python run_pipeline.py` runs these 8 steps in order.

Step	What happens	What comes out
1. Download	Fetch the 8 raw CSV files and check each one by checksum and row count.	<code>data/raw/</code> (550,759 rows in 8 tables)
2. Load	Read every file into DuckDB as plain text so bad values survive.	<code>raw_*</code> tables
3. Validate	Run 81 checks from the contract and from SQL rule files.	<code>dq_exceptions</code> : 2,483 rows, none deleted
4. Reconcile	Per order: sold vs paid, gap, then assign a bucket.	<code>reconciliation</code> table and evidence CSVs
5. Star schema	Build 3 fact tables and 4 dimension tables.	<code>data/processed/</code> CSV + Parquet
6. KPIs + forecast	Run the KPI queries and the forecast.	<code>reports/metrics.json</code>
7. Reports	Build the Excel and the dashboard.	<code>summary.xlsx</code> , <code>dashboard.html</code>
8. Docs	Write README, memo, notes, data dictionary from the numbers.	markdown files

The most important idea: `metrics.json` is the **single source of truth**. Every number in every document comes from it. That is why you can quote any number from the README in an interview and it is real.

The mismatch buckets, in plain words

Before naming any bucket I looked at the mismatches in the data and found patterns. Each bucket exists because the data showed that pattern. The files in `reports/evidence/` are the proof.

Bucket	Orders	Gap (R\$)	In plain words
matched	98,362	n/a	Sold equals paid within 1 cent. Nothing to explain.
missing_items	775	162,591.95	A payment exists but no items. Mostly canceled or unavailable orders. The money question: was it refunded?
installment_interest	232	3,050.01	Customer paid by card in installments and paid extra. The extra follows a fixed rate table that grows with the number of installments. The order list never recorded it.
missing_payment	1	143.46	Items exist but no payment at all.
untracked_discount	15	123.19	Customer paid exactly 5% or 10% less than the price. The discount lives only in the payment.
unexplained	5	77.54	Fit none of the rules. A person has to look.
voucher_related	7	17.48	Paid by card plus voucher and the total overshoots the order.
rounding	44	1.00	A few cents off on orders with many items. Each line rounds on its own. Harmless.

Rules run top to bottom and the first match wins, so an order never lands in two buckets.

What the numbers mean for a manager

- Money at risk is concentrated: 344 orders (31.9% of the unmatched ones) hold 80% of the unmatched money. Start there.
- Customers notice: unmatched orders average 2.33 stars against 4.11 for matched ones. Careful: mostly because those are cancellations. Correlation, not proof.
- Forecast: the naive baseline (`naive_last_value`) won with 8.74% error. The fancier models lost. Say that out loud in interviews. Honesty about a model losing is a strength.

How to study this (do one block per day)

Day	Time	Do this	Check yourself
1	30 min	Read the 5 sentences and the words page. Say the 5 sentences out loud.	Can you explain the project to a friend in 60 seconds?
2	1 hour	Read Part 4 of the notes (rebuild by hand), piece 1, line by line.	Why start from all order ids? Why sum each side before joining? Why DECIMAL?
3	1 hour	Read the 3 failure-injection tests in Part 1 under tests/ .	What does "input = clean + quarantined" prove?
4	1 hour	Read the 15 interview questions in Part 3. Answer each out loud before reading the answer.	Which 3 do you still stumble on? Repeat only those.
5	1 hour	Close everything and rewrite the reconciliation SQL from memory.	Do you get 99,441 rows and 98,362 matched?

Rules for yourself:

- If you cannot explain a decision, find it in the Decisions table (Part 2) and read why we did it.
- Never say a number in an interview that is not in this guide.

Part A: Full learning guide (from NOTES.md)

This file is for you, not for recruiters. Read it top to bottom once, then use the interview section as flash cards. Every number here is pulled from `reports/metrics.json`, so if you quote it in an interview, it is real.

How to read the project in 30 minutes:

1. `run_pipeline.py` (the whole flow on one screen)
2. `config/contracts.yaml` (what "good data" means)
3. `sql/reconciliation/01_order_totals.sql` and `02_categorize.sql` (the core)
4. `reports/evidence/*.csv` (why the buckets exist)
5. `tests/test_validation.py` (how we prove the checks work)

Part 1: file by file, function by function

`run_pipeline.py`

- **What:** the single entry point. Runs 8 steps in order: download, load, validate, reconcile, star schema, KPIs + forecast, reports, docs.
- `run(paths, download)`: does the work and returns the metrics plus the open DuckDB connection (tests use the connection to query intermediate tables).
- `main()`: command-line flags `--raw-dir`, `--out-dir`, `--no-download`.
- **Why this way:** one command rebuilds everything from scratch, so results are reproducible. Tests call `run()` directly on the fixture.
- **Alternative considered:** Airflow/Prefect/dbt. Overkill for one machine and one dataset; a plain script is easier to read and to explain. dbt would be the next step in a team setting.
- **Interview Q:** "How would you schedule this?" Cron or a GitHub Actions schedule calling `python run_pipeline.py`, alert on the match rate and on high-severity exceptions. In a company: Airflow or dbt Cloud, same SQL files.

`src/download.py`

- **What:** gets the 8 CSVs and proves they are the real dataset.
- `ensure_raw_data(raw_dir)`: if files are missing or wrong, tries Kaggle (only with credentials), then a GitHub mirror pinned to a commit. Then calls `verify()`.
- `verify(raw_dir)`: SHA-256 checksum and row count for every file. Any mismatch stops the pipeline.
- `sha256()`, `is_complete()`, `_from_kaggle()`, `_from_mirror()`: helpers.
- **Why:** Kaggle was blocked on the build machine. A random mirror could be edited (one was: its items file was missing a column). Pinning to a commit + checksums means the input can never change silently.
- **Interview Q:** "How do you know your data is the real dataset?" Row counts match the known dataset (99,441 orders, 112,650 items, 103,886 payments), two independent mirrors had identical checksums, and the pipeline re-checks the checksums every run.

src/config.py

- **What:** paths, the raw file names, and the match tolerance (R\$ 0.01).
- **Paths:** a dataclass so tests can point everything at a temp folder.
- **Why:** one place to change settings. No magic strings spread across files.

src/load.py

- `load_raw(con, raw_dir)`: reads each CSV into DuckDB as `raw_<table>`, **every column as text**, and adds `_row_id` (1 = first data row).
- **Why text:** if a price column contains `abc`, a typed load would crash or silently turn it into NULL. Loading as text keeps the bad value so a check can report it.
- **Why `_row_id`:** natural keys can be duplicated (the review file reuses 814 review ids). A row number is the only id that is always unique, so the exceptions table points at `_row_id`.
- **Alternative:** pandas `read_csv`. Fine at this size, but DuckDB is faster, speaks SQL, and shows the SQL skill.

config/contracts.yaml

- **What:** the schema contract. Per table: columns, type, required or not, allowed values, primary key, foreign keys, severity overrides.
- **Why YAML:** a business person can read and edit it. Adding a rule is a config change, not a code change.
- **Alternative:** Great Expectations or Pandera. Good tools, but they hide the logic. Here every rule is plain SQL you can explain line by line.

src/validate.py

- `load_contracts()`: reads the YAML.
- `build_staging(con, contracts)`: creates `stg_<table>` with `TRY_CAST(col AS type)` per contract. Bad values become NULL in staging but are caught by the type check.
- `generic_checks(contracts)`: fills the SQL templates in `sql/checks/generic/` for every table and column: duplicate key, required null, type mismatch, allowed values, orphan foreign key.
- `business_checks()`: loads every `sql/checks/*.sql` file.
- `run_validation(con)`: runs all 81 checks as one `UNION ALL` into `dq_exceptions`, then builds `dq_row_status` (clean / warning / quarantined) and the `clean_<table>` views.
- **Why:** checks are data, not code. Same six output columns for every check, so they stack into one table.
- **Key rule:** nothing is deleted. Quarantined rows stay in `stg_` and are listed in `dq_exceptions`.
- **Interview Q:** "Why not just drop bad rows?" Because then the totals silently change and nobody can audit why. Routing them to an exceptions table keeps the count honest: rows in = usable + quarantined, and a test proves it.

sql/checks/generic/*.sql

- `duplicate_key.sql`: `ROW_NUMBER() OVER (PARTITION BY key ORDER BY _row_id)`; every copy after the first is flagged. Window function, one pass.
- `required_null.sql`, `type_mismatch.sql`, `allowed_values.sql`: simple `WHERE` filters on the raw text.
- `orphan_foreign_key.sql`: `NOT EXISTS` against the parent table. `NOT EXISTS` is safer than `NOT IN` because `NOT IN` returns nothing if the subquery has a NULL.

sql/checks/*.sql (business rules)

- `orders_date_sequence.sql`: approved/shipped/delivered before purchase (high), delivered before carrier (medium), shipped before approval (low).
- `orders_status_date_conflict.sql`: status says delivered but no date, or a date on an undelivered order.
- `items_non_positive_amounts.sql`: price ≤ 0 , freight < 0 .
- `payments_value_checks.sql`: negative, zero, and < 1 installment payments.
- `payments_sequence_gap.sql`: `LAG()` finds holes in `payment_sequential`.
- `reviews_unknown_order.sql`, `reviews_date_sequence.sql`.
- **Why severities differ**: high = money or counts wrong if you trust the row; medium = suspicious; low = informational. Example: shipped before approval happened 1,359 times; it is normal when approval is recorded late, so it is low.

sql/reconciliation/01_order_totals.sql (the core)

- **What**: one row per `order_id` with sold (items) and paid (payments) totals and the gap.
- **CTEs**: `universe` (every `order_id` from all 3 tables, `UNION`), `sold`, `paid`, `first_payment`, `dq_orders`, `orders_one`.
- **Why a universe CTE**: if you start from `orders` and `LEFT JOIN`, a payment whose order is missing disappears. Starting from the union of all ids means orphans show up.
- **Why aggregate before joining**: joining items to payments row by row multiplies rows (3 items x 2 payments = 6 rows) and doubles the money. Aggregating each side to one row per order first avoids the fan-out.
- **Why DECIMAL**: floats cannot store 0.1 exactly; summing thousands of prices drifts. `DECIMAL(12,2)` keeps cents exact.

sql/reconciliation/02_categorize.sql

- A `CASE` evaluated top to bottom; the first rule that matches wins, so each order gets exactly one bucket. Then a second `CASE` sets severity.
- Order matters: rounding is checked before interest so a 2-cent gap on a 6-item card order counts as rounding, not interest.

sql/reconciliation/03_gap_rankings.sql

- `DENSE_RANK() OVER (PARTITION BY purchase_month ORDER BY abs_gap DESC)`: biggest gap per month.
- Same per state.
- `SUM(abs_gap) OVER (ORDER BY abs_gap DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) / SUM(abs_gap) OVER ()`: running share of all unmatched money (Pareto).
- `LAG(unmatched_orders) OVER (ORDER BY purchase_month)`: change vs previous month.
- **Interview Q**: "RANK vs DENSE_RANK vs ROW_NUMBER?" `ROW_NUMBER` always 1,2,3. `RANK` skips after ties (1,1,3). `DENSE_RANK` does not skip (1,1,2). `DENSE_RANK` here because ties are real (same gap) and "the 2nd biggest gap" should mean the 2nd distinct value.

sql/reconciliation/04_evidence.sql

- Named queries that produce the proof for each bucket, saved to `reports/evidence/`. This is what you show when someone asks "how do you know it is interest?".

src/reconcile.py

- `run_reconciliation(con, evidence_dir)`: runs the 3 SQL files, builds `recon_exceptions` (one per unmatched order), `all_exceptions` (data quality + reconciliation), `exceptions_summary`, `reconciliation_summary`, and writes the evidence CSVs.
- `CATEGORY_RULES`: the bucket list with rule, evidence and action. Loaded into DuckDB so it joins onto the summary.
- `_run_sql_file()`: fills `$tolerance` into the SQL.

sql/star/star_schema.sql **and** src/star_schema.py

- **What:** 3 facts (`fact_orders`, `fact_reconciliation`, `fact_exceptions`) and 4 dims (`dim_date`, `dim_customer_state`, `dim_payment_type`, `dim_product_category`), exported to CSV and Parquet.
- **Why a star:** Power BI works best with narrow dimensions filtering long facts over single-direction one-to-many relationships. Slicers on a dim filter all facts at once.
- **Choices:** order category = the category of the most expensive item (`ROW_NUMBER` by price); main payment type = the biggest payment; unknown members (`UNKNOWN`, `unknown`, `none`) so no fact key is ever blank.
- `build_star_schema(con, out_dir)`: runs the SQL and exports with `COPY ... TO`.

sql/kpis.sql **and** src/metrics.py

- Every KPI is a named SQL query. `compute_kpis()` runs them all; `build_metrics()` puts them in one dict with a few derived numbers; `write_metrics()` saves `reports/metrics.json`.
- **Why one JSON:** a single source of truth. The Excel, the dashboard and every doc read from it.

src/forecast.py

- `monthly_orders()`: orders per purchase month.
- `complete_window()`: keeps the longest run of months with at least 10% of the median volume. That drops the test-like 2016 months and the near-empty final months.
- `naive_last()`, `seasonal_naive()`, `moving_average_3()`, `holt_damped()`: the candidates.
- `mape()`: mean absolute percentage error.
- `run_forecast()`: last 3 complete months are the holdout; score each model; refit the winner on everything; forecast the next months. Writes a verdict that says honestly which kind won.
- `forecast_table()`: long table for Power BI.
- **Why a baseline:** a model is only useful if it beats "same as last month". Here it did not: `naive_last_value` scored 8.74% vs 10.87% for the best model.

src/report_excel.py

- `build_excel()`: KPI sheet (value + plain definition), gap by category, exceptions (summary + every row), reconciliation (every unmatched order, ranked), monthly, states, categories, forecast. Currency formats in R\$, frozen headers, filters, severity colors.
- `_write_table()`, `_autosize()`, `_fmt_for()`: formatting helpers.

src/report_html.py

- `build_dashboard()`: one self-contained HTML file (Plotly inlined, works offline): stat tiles, 7 charts, top-15 gap table, light and dark mode.
- `_hbar()`, `_layout()`: shared chart styling. Colors are from a palette checked for colorblind safety.

src/render_docs.py

- `render()`: replaces placeholders (a metric path and a format inside double curly braces) with values from metrics.json; `table`: placeholders build markdown tables. Missing metric = error (in the docs test).
- **Why**: you asked that no number is ever typed by hand. This makes it impossible, and `tests/test_docs.py` fails if someone edits README.md by hand.

src/data_dictionary.py

- `build_data_dictionary()`: reads real column names and types from DuckDB and joins them to descriptions. A column without a description fails the pipeline.

src/sqlutil.py

- `named_queries(path)`: splits a .sql file on `-- name: x` lines.

tests/

- `conftest.py`: `RawCopy` copies the fixture to a temp folder and lets a test append/update rows; `clean_run` runs once on untouched data.
- `test_validation.py`: injects one bad thing per test (duplicate order, duplicate payment, orphan payment/item, negative/zero price, out-of-order dates, payment mismatch, bad type, unknown status, required null, unknown review order, sequence gap) and asserts the exact check and severity.
- `test_completeness.py`: input = clean + quarantined, and every order lands in the reconciliation exactly once, also after corruption.
- `test_reconciliation.py`: builds an order for each bucket and checks it lands there.
- `test_pipeline.py`: whole pipeline on the fixture; files exist; every fact key finds its dimension.
- `test_docs.py`: committed docs equal a fresh render from metrics.json.
- `scripts/make_fixtures.py`: builds the fixture from real rows so every bucket is represented.
- **Proof the tests bite**: during the build, one check file was removed and one rule disabled on purpose; the matching tests failed, then passed again when restored. Do this yourself once (Part 4).

Part 2: Decisions (every judgment call)

Decision	Why	What I gave up
DuckDB, not pandas, for the heavy lifting	SQL is the skill employers test; DuckDB is fast and needs no server	pandas-only folks need to read SQL
Load raw as text	bad values become exceptions instead of crashes or silent NULLs	one extra staging step
<code>_row_id</code> as record id	natural keys are duplicated in the raw data	ids are file-position based; re-sorting the file changes them
Never delete, quarantine	auditability; totals stay explainable	clean views must be used deliberately
Reconcile on all staged rows, not the clean views	reconciliation must see both systems as they are; a duplicate payment should show as a mismatch, not vanish	DQ problems also show up as recon gaps (flagged by <code>has_dq_exception</code>)
Universe = union of order_ids from all 3 tables	orphans cannot hide behind a join	slightly more complex SQL
DECIMAL for money	exact cents	none worth mentioning

Decision	Why	What I gave up
Tolerance R\$ 0.01	business requirement; one cent	multi-item rounding needed its own bucket
Investigate before bucketing	buckets must be evidence, not guesses	took longer
Rounding = up to 1 cent per item	each line is rounded on its own; all small gaps were on multi-item orders	a single-item 2-cent gap is "unexplained" (strict on purpose)
Interest = card only, 2+ installments, overpaid	paid/sold ratio sits on fixed multipliers that rise with installments	a few orders with odd ratios are still called interest (69.83% sit on a shared ratio)
Discount = exactly 5% or 10% of product price, +/- 2 cents	the shortfalls land on those two values	other discount sizes fall to unexplained
Missing items on canceled/unavailable = medium, else high	canceled orders with payments are likely refunds; open orders with no items are a real problem	refunds are not in the data, so medium is a judgment
Payment sequence gap = low	79 of 80 such orders still reconcile	a real missing payment with a gap is still caught by reconciliation
Review id duplicate = medium, not high	the reviews are real, only the id is reused	none
Order category = most expensive item	one category per order for slicing	multi-category orders are simplified
Forecast window = longest run of complete months	edge months are near-empty and would wreck any model	fewer training months (17)
Keep the baseline when it wins	honesty; a worse model is worse	less impressive-sounding
Plotly single HTML file	anyone can open it, no server, no Power BI license	a few MB, because the charting library is inlined
Docs rendered from metrics.json	no hand-typed numbers, ever	template syntax to learn
Large fact CSVs not committed	repo size; Parquet copies are committed	CSV users must run the pipeline

Part 3: 15 interview questions about this project

1. Walk me through the project in one minute.

Two systems disagree: order items say what was sold, payments say what was paid. I loaded 550,759 raw rows from 8 tables into DuckDB, ran 81 data quality checks without deleting anything, and reconciled 99,441 orders. 98.91% match to the cent. I investigated the 1,079 that don't, found evidence-based causes for all but 5, and the big one is R\$ 162,591.95 paid on orders with no items. Output: star schema for Power BI, Excel, a dashboard, and a one-page memo.

2. How did you define a match?

Paid minus sold within R\$ 0.01, per order, with sold = item price + freight and paid = sum of payments. Money stored as DECIMAL so cents are exact.

3. How did you decide the mismatch categories?

I looked first. Overpaid card orders had paid/sold ratios sitting on fixed values that rise with installments (median 3.46% over at 2 installments, 15.50% at 12). That is an interest table. Underpaid orders were short by exactly 5% or 10% of the price, which is a discount. Small gaps were all on multi-item orders, which is rounding. Each rule has a CSV of evidence behind it.

4. What was the biggest finding?

775 orders have payments but no items: R\$ 162,591.95, 97.94% of all unmatched money. 98.97% are canceled or unavailable, so the question for finance is whether those customers were refunded.

5. How do you know you did not lose any rows?

Tests. For every table, input rows = usable rows + quarantined rows, and every order_id from any source appears exactly once in the reconciliation. Those tests run on clean data and again after injecting duplicates, orphans, negative prices and bad types.

6. How did you test the checks?

Failure injection. Each test copies a small real fixture, plants one known bad record (duplicate order, orphan payment, negative price, delivered before purchase, payment off by R\$ 37) and asserts the exact check and severity fire. I also broke a check on purpose to confirm the tests fail.

7. Why not delete bad data?

Deleting changes totals with no audit trail. I route every failing row to an exceptions table with a reason and severity; the clean views exclude high/medium rows, but the raw and staged data are untouched.

8. Give an example of a data quality issue you found.

166 orders were handed to the carrier before they were purchased, which is impossible, so high severity. 814 review rows reuse a review id across orders, medium. 1,359 orders shipped before payment approval was recorded; that is plausible, so low.

9. Show me a window function you used and why.

`DENSE_RANK() OVER (PARTITION BY purchase_month ORDER BY abs_gap DESC)` to rank the largest gaps each month, and a running `SUM() OVER (ORDER BY abs_gap DESC)` divided by the total to get a Pareto curve: 344 orders (31.9% of unmatched) hold 80% of the unmatched money. That tells the team where to start.

10. How did you avoid double counting when joining items and payments?

Aggregate each side to one row per order in its own CTE, then join. Joining raw rows would multiply items by payments.

11. Tell me about the forecast.

Monthly orders, holdout of the last 3 complete months, four candidates. Honest result: a naive baseline wins. `naive_last_value` beats every model on the holdout, so the baseline is the forecast. A model that loses to 'same as last month' is not worth shipping. `naive_last_value` had 8.74% MAPE; the best model had 10.87%. In the holdout table, Holt's trend predictions run above the actuals because volume flattened in 2018, and seasonal naive copies 2017, a ramp-up year. Whatever wins is what ships, and the docs say which kind won.

12. What would you do differently with more time?

Pull refund data to close the missing-items question, get a second year for seasonality and a Black Friday flag (November 2017 was 62.9% above October), move the SQL into dbt with tests, and schedule a daily run with alerts.

13. Is there a customer impact?

Yes. Unmatched orders average 2.33 stars vs 4.11 for matched orders, and 61.9% of their reviews are 1 or 2 stars. Late deliveries show the same pattern: 2.27 vs 4.29. Correlation, not proof of cause: the missing-items orders are mostly cancellations, and customers rate cancellations badly.

14. How would a manager use this?

The memo gives three actions. The Excel has the KPI sheet with definitions and the full list of unmatched orders ranked by money, so a finance analyst can work top-down. The Power BI spec rebuilds the dashboard on the star schema with measures that must equal metrics.json.

15. What are the limits of your conclusions?

No refund or bank settlement data, so "missing items" cannot be closed here. Buckets are inferred from strong patterns, not confirmed by Olist. The interest rule catches some orders whose ratio is not on the shared table. The forecast has 17 training months.

Part 4: Rebuild by hand (prove you understand it)

Close this repo and write these three from memory in a blank DuckDB session. Then compare.

1. The reconciliation query (the most important one)

Goal: one row per order with sold, paid, gap and a match flag, without losing orphans or double counting.

```
with universe as (
  select order_id from stg_orders
  union
  select order_id from stg_order_items
  union
  select order_id from stg_order_payments
),
sold as (
  select order_id, count(*) as item_count, sum(price + freight_value) as expected_total
  from stg_order_items
  group by order_id
),
paid as (
  select order_id, sum(payment_value) as paid_total,
    max(payment_installments) as max_installments,
    bool_or(payment_type = 'voucher') as has_voucher,
    bool_and(payment_type = 'credit_card') as only_credit_card
  from stg_order_payments
  group by order_id
)
select
  u.order_id,
  s.expected_total,
  p.paid_total,
  coalesce(p.paid_total, 0) - coalesce(s.expected_total, 0) as gap,
  abs(coalesce(p.paid_total, 0) - coalesce(s.expected_total, 0)) as abs_gap,
  case
    when p.paid_total is null then 'missing_payment'
    when s.expected_total is null then 'missing_items'
    when abs(p.paid_total - s.expected_total) <= 0.01 then 'matched'
    else 'mismatch'
  end as status
from universe u
left join sold s using (order_id)
left join paid p using (order_id);
```

Check yourself: you should get 99,441 rows and 98,362 matched. Then add the bucket `CASE` from `02_categorize.sql` yourself.

2. The duplicate check with a window function

```
select _row_id, order_id,
  row_number() over (partition by order_id order by _row_id) as occurrence
from raw_orders
qualify occurrence > 1;
```

Then inject a duplicate (copy a row in the CSV) and watch it appear. Explain why `occurrence > 1` and not `count(*) > 1` (you want to flag the extra copies, not the original).

3. The completeness test

In Python: load a table, run validation, and assert `raw rows == clean rows + quarantined rows`. Then break it on purpose: change the clean view to also drop warning rows and watch the test fail. That is the whole idea of "nothing silently dropped".

Part 5: Resume bullets (all numbers computed by the pipeline)

- Built an end-to-end SQL/Python reconciliation pipeline (DuckDB, pandas) matching sold vs paid values for 99,441 e-commerce orders; 98.91% matched to the cent and 98.3% of value mismatches were explained by evidence-based rules.
- Surfaced R\$ 162,592 in payments on 775 orders with no items recorded (97.9% of the unmatched value) and showed 344 orders hold 80% of the gap using SQL window functions.
- Designed 81 config-driven data quality checks over 550,759 rows, routing 2,483 exceptions to an audit table by severity with zero deleted records, verified by failure-injection tests in GitHub Actions CI.
- Modeled a Power BI star schema (3 facts, 4 dimensions) and delivered an Excel KPI report, an interactive dashboard and a one-page memo; linked unmatched orders to lower satisfaction (2.33 vs 4.11 stars).
- Benchmarked monthly demand forecasts against naive baselines on a 3-month holdout and shipped the winner, `naive_last_value` (8.7% MAPE vs 10.9% for the best non-baseline model).

Rule for yourself: never say a number in an interview that is not in `reports/metrics.json`.